

Lecture X - Git and Your Project

Programming with Python

Dr. Tobias Vlček

Kühne Logistics University Hamburg - Fall 2026

Checkpoint 5

Sessions VIII-IX. The first 40 minutes are the checkpoint. It starts now: the board runs one final audit before the big news.

- **Individual work:** no neighbors, no messaging
- **Any AI tool is allowed:** being able to **VERIFY** its output is the skill being graded
- The **link is on Moodle**. Open it and start
- 6 short tasks: the right number out of a pandas table, a line the AI got wrong, a correct read of a chart
- It sweeps **Sessions VIII-IX:** pandas filtering and `groupby`, fair charts vs. misleading ones

When you're done

Cmd/Ctrl+S, then menu → *Download* → *Download Python code* → upload the `.py` to the “**Checkpoint 5**” assignment on Moodle, before the 40 minutes are up. **No retakes:** one sitting.

...

Note

The live checks under the tasks are **provisional**. The final grading runs on my side. The two multiple-choice tasks only say “*recorded*”, never correct or wrong. That’s expected, they’re scored later. Everything in it was rehearsed in the labs.

EP 10: The Exit

The deal closes

Pens down. The checkpoint is behind you, and so, it turns out, is the startup. Overnight the rival chain **MunchCorp** stopped competing and made an offer: they’re **buying the company**. The investor read the term sheet, did the math against her own, and told you to take it: her money would have bought growth, theirs buys the whole shop.

...

The lawyers will tell you deals close on the numbers. This one closed on **yours**. The clean order data you pulled together in the data room (Session VIII) and the pitch deck you built from it (Session IX) are exactly what convinced the acquirer the books were real.

...

No cliff-edge growth chart with a chopped axis. Just data that survived a second look. **That's why they signed.**

The handover

Tobi's job today is to hand over the **data room**: the folder with every file the company ever built. You know that folder. It's `python-labs`, two labs later.

...

MunchCorp's lawyer opens it and asks two questions:

- `growth.png`: is this the chart the investor **saw**, or has it changed since?
- `orders_messy.csv`: **who** changed it after Tobi's export, and **what**?

...

Nobody can say. A folder shows what is in it **now**, and nothing about how it got there. Today you give a folder a **history**, with a tool called git.

Your exit package

Every acquisition comes with an exit package. Here's yours: you don't join MunchCorp. You **spin off your own idea** and build it.

...

- The tools are the ones professionals use: uv and Zed have been on your laptop since Session VII, **git and GitHub** join them today
- The work is a real project, built with a partner, over the next weeks
- Today we set up the repo; the building starts here

...

Let's get you kitted out.

The Project

The brief

Over the next weeks you build **one real project** and present it in the final sessions.

- Work in **pairs**: solo is fine if the numbers don't come out even
- Demos run across **Sessions XII and XIII**: 10 minutes, plus 5 minutes for questions
- **Deliverable**: a **GitHub repository link**, handed in on Moodle **before your own demo**
- The repo's `README.md` carries a short **AI-disclosure section**: which tools you used, what for, and what you **verified yourself**

...



Tip

Pick something you're **actually interested in**. You'll spend real hours on it. Curiosity is the fuel that carries a project past the boring middle.

How it counts

- Project **and** presentation together are worth **40 of the 100** course points, judged on scope, work and presentation. Both partners get the same points
- Your **commit history counts**: it's the evidence of *how* the work happened, which the final state alone can't show. Commit small and often (git block, shortly)
- **Sessions XI and XII are supervised work sessions**: I'm in the room while you build, so bring your questions

...



Note

The project does not have to be flawless, and an unfinished one can be presented too. Show that you scoped it realistically, built it in the open, and can explain your own code. That's the whole bar.

Idea 1 & 2

Data analytics dashboard

- **Core**: load and clean a real CSV with pandas, answer **3 business questions** with charts

Bonus: interactive dashboard (streamlit), automated reporting.

...

Web scraping pipeline

- **Core**: scrape **one** site politely into a CSV and analyze it

Bonus: multi-source, change detection, alerts.

Idea 3 & 4

Simulation study

- **Core**: a Monte Carlo simulation of a business decision (pricing, staffing, inventory) with matplotlib results

Bonus: interactive parameters, scenario comparison.

...

Automation assistant

- **Core**: a script that does a real repetitive task **end to end**, with error handling and logging

Bonus: scheduling, notifications, a small UI.

Idea 5 & 6

Game

- **Core:** a complete playable **terminal or pygame** game with save/load

Bonus: levels, high scores, procedural content.

...

ML-powered app (*ambitious: lean on AI*)

- **Core:** train a **simple** model on tabular data and wrap it in a small interface

Bonus: monitoring, A/B comparison.

Idea 7 & 8

Your startup spin-off

- **Core:** take any system from the course universe (courier routing, review moderation, demand forecasting) and build the **real** version

Bonus: wherever your version wants to grow.

...

Your own idea: the open door

- Have something else in mind? **Make it ambitious.** Bring it and we'll discuss scope and feasibility together.

Form your pair

One rule before you pair up

You build your project with the agent, and the agent lives in Zed. So **every pair needs a laptop that runs Zed.**

- Zed doesn't run on an iPad, so **two iPads can't form a pair**
- Only an iPad? Pair with someone who has a laptop. You can still work on the project: in a GitHub codespace, or on that laptop

Take ~25 minutes

Right now, in the room:

- **Find your partner** (or decide to go solo if the count is odd). No partner yet? **Come to the front**, I'll match you
- **Pick a direction** from the menu, or bring your own
- Talk it through: what's the *core* you can definitely finish, what's a *bonus* if time allows?

...



Tip

A good outcome by the end of this block: a one-sentence pitch (“we’re building X that does Y for Z”), a name for the project, and a decision on who owns the shared repo (the owner creates it in the git block, so pick whoever did the git homework; the partner clones). The sentence keeps you honest about scope; the name goes on the repo in a minute.

Your First Repo

Check your homework

git and gh were the homework from Session IX. Before we build anything, they need to work. In Zed’s terminal (`Ctrl+``):

```
git --version
gh --version
gh auth status
```

- Two version numbers, and the last command says **Logged in to github.com?** You’re set.
- No login yet? The next slide is yours.

...

! Important

Didn’t work? Don’t burn the session fighting it. Tell me, pair up with your partner’s working machine, and follow along on the slides for now. I sort it out with you in class or by e-mail; nobody gets left behind.

Once per machine

Before your laptop can talk to GitHub, it has to prove it’s allowed to. Signing in to **Zed** **does not do this**; git needs its own login, and the gh tool handles it. If `gh auth status` showed no login, do it now, once per computer:

```
gh auth login
```

Choose GitHub.com, **HTTPS**, authenticate in the browser, and answer yes to “Authenticate Git with your GitHub credentials?”.

...

! Important

Login fighting you? Same rule: **tell me**, follow along on the slides, and finish at home with the [Git Basics](#) page, or with me in the supervised sessions. Nobody's project stalls on a login.

Tell git who you are

Everyone, logged in or not: three settings, once per computer. Running them twice does no harm.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global pull.rebase false
```

- **Name and e-mail** are how git answers the lawyer's *who*? Use the e-mail of your GitHub account, so GitHub knows it's you
- **The third line** is for pair work later: git then simply combines your partner's changes with yours

Your project starts on GitHub

Everyone, right now, a **practice repo**. On github.com click **New repository**, then:

- **Name:** `python-practice`
- **Private**
- Tick **Add a README file**
- **.gitignore template:** `Python`

Create repository, then copy the **HTTPS URL** from the green **Code** button: `https://github.com/you/python-practice.git`

...

That's a **repository**: a folder git watches, with its whole history. So far it lives only on GitHub. Next we bring it down.

Bring it down: clone

In Zed's command palette (`Cmd/Ctrl+Shift+P`): **git: clone**, paste the URL, and pick the folder that `python-labs` sits in, so the repo lands **next to** `python-labs` and not inside it. Zed opens the new folder as your workspace: the file tree on the left shows `README.md` and `.gitignore`.

The same in the terminal. Use one way, not both:

```
git clone https://github.com/you/python-practice.git
```

...

Note

Everything from here on happens **inside this folder**: git, uv, the AI. If Zed ever shows an empty file tree, **File → Open Folder** and pick the cloned folder itself, not the one above it.

Make it a Python project: `uv init`

Open Zed's built-in terminal (`Ctrl+``); it starts inside your workspace. Then:

```
uv init --no-package
uv run python main.py
```

`uv init` adds `main.py`, `pyproject.toml` and `.python-version` next to the `README.md` and `.gitignore` that GitHub made. The second line runs `main.py` the way you run a lab, and prints `Hello from python-practice!`

...

Note

`--no-package` keeps it a flat folder of scripts, like `python-labs`, which the same command made in Session VII; without it, uv builds an installable package layout you don't need yet. And no `git init`: the folder came from GitHub, it **is** a git repository already.

Same agent, new folder

Part III **encourages** AI, and your agent is already here: Mistral Vibe works in whatever folder Zed has open. Open the agent panel and ask:

*Explain what `main.py` does, then add a one-line comment at the top that says so.
Don't change anything else.*

This time, **watch the panel while it works**: what does Vibe do before it shows you a change? Then read the diff before you accept it, as always.

...

Important

The disclosure rule applies from today: your repo's `README.md` says **which tools you used, what for, and what you checked yourself**.

Inside the panel: a model in a loop

- Mistral Vibe is an agent: the language model you met in Session VI, wired into a loop with tools
- The model **proposes** an action: read a file, edit a line, run a command
- The tool **runs** it, the result **feeds back**, and it proposes again, until the goal is met
- No magic, and no memory between sessions

...

Every failure mode from Sessions VI and VIII applies at every step, and a loop gives them many steps to compound in. The diff you just read is the guardrail.

Git

The five words

Git has a small vocabulary. Learn these five and you can do everything this project needs:

- **Repository**: a folder git watches, with its whole history
- **Commit**: one saved snapshot, with a short message
- **Push**: send your commits up to GitHub
- **Pull**: bring your partner's commits down to you
- **Clone**: make your own copy of a repo from GitHub

...

Two of them you've already done: the repo exists, and you cloned it. The other three now, **live, in your practice repo**. Mistakes are cheap when the repo is five minutes old.

Your first commit

Your folder has new files worth saving: `main.py` with its new comment, `pyproject.toml`, `.python-version`, and `uv.lock`, the exact package versions, so every laptop installs the same ones. In Zed's **command palette**:

- `git: stage all`: mark everything as part of the snapshot
- `git: commit` (`Cmd/Ctrl+Enter`): type a message, save it

...

The same thing in the terminal, the words you'll see everywhere:

```
git add .
git commit -m "Set up the project"
```

A commit lives **only on your computer** so far. Next we send it to GitHub.

Push

Because the folder came from GitHub, git already knows where "up" is. In Zed: `git: push`. In the terminal:

```
git push
```

...

Reload the repo page on github.com: `main.py` is there. That's the commit cycle you'll repeat all project long: **edit** → **commit** → **push**. Pull is the other direction; you'll see it in a minute.

Your history

Two commits in, the folder has a history. Read it:

```
git log --oneline
```

One line per commit, newest first: a short ID and the message. Yours sits on top of GitHub's `Initial commit`. On github.com, **Commits** (top right of the file list) shows the same list with author and time; click a commit to see what it changed.

...

That answers the lawyer for every file in the repo: **who** changed it, **when**, and **what it looked like before**. It's also the list I read when I grade your project. `Set up the project` tells me something; `stuff` doesn't.

The real project repo: owner only

Now the one that counts. **Repo owner only**, same recipe as the practice repo, with four extras:

1. Name it after your **pitch**; **Private**, README, Python `.gitignore`
2. **Settings** → **Collaborators**: invite your partner and me (my GitHub username is on Moodle), so I can read it for grading
3. Clone it in Zed, `uv init --no-package`, commit, push, exactly as before
4. Needs pandas or matplotlib? `uv add pandas matplotlib` in this folder: every project has its own packages

...

Partner: **don't create a second project repo**. Yours arrives by invitation in a moment.

Your partner joins

The **partner** accepts the invitation (GitHub e-mails it) and clones the project **into a fresh folder**, next to the practice repo:

- In Zed: `git: clone`, paste the project's **HTTPS URL**

Or in the terminal:

```
git clone https://github.com/owner/project.git
```

...

Cloning brings the whole project, its history and the connection to GitHub; the partner pushes and pulls normally from the first moment.

The fifth word: pull

1. The **owner** writes your one-sentence pitch into `README.md`, with a heading `## AI disclosure` below it, then commits and pushes
2. The **partner** runs `git: pull` (`git pull --no-edit`): the pitch lands on the second laptop
3. **Swap:** the partner adds a line under the heading, commits and pushes, and the owner pulls

...

Both of you have now pushed and pulled in the real repo. On github.com, **Commits** shows both names.

...

i Note

Clock ran out? This step can wait: inviting and cloning works just as well at the start of **Session XI**, with me in the room.

How we work together

Two people, one repo. Four habits keep it painless:

- **Pull before you start.** `git: pull` first, every session: this alone prevents almost every problem
- **Commit small and often.** Little snapshots with clear messages beat one giant one
- **A conflict? Call me.** If git marks a clash, don't fix it alone in week one. Bring it to class
- **Never force anything.** No `--force`, no "force push", ever

...

i Note

Everything today (the actions, the terminal equivalents, the setup, and a **cheatsheet table**) lives on the [Git Basics](#) page. When today's follow-along is a fading memory, that's the page you reopen.

Send-off

The road to the finish

- **Session XI:** work session, plus one short segment: why AI text now carries a watermark, and why that makes your README disclosure standard practice
- **Session XII:** work session, plus teaching your agent your project's rules. **The first demos happen here**

- **Session XIII:** the remaining presentations, and where this course hands you over to your thesis

...

Tip

Between now and then, the repo tells your story. Commit as you go. A steady history is worth more than a heroic final night.

Demo Day

Format

- **10 minutes** per pair presenting, **5 minutes** of questions
- The first pairs present at the end of Session XII, the rest in Session XIII
- Order: drawn at random at the start of each demo session, with the `random` module from Session VI

What to show

- **The thing running.** The output, and the command that produces it
- **How you worked.** Open the repo: the commit history instead of describing it
- **Your AI disclosure.** Which tool, what for, what you verified yourself
- **What went wrong.** Where you had to step in, what cost you an hour

...

Your machine, your repo, your demo: it must run from the clone on the room's projector.

The reference pages

Note

The [Git Basics](#) page is the reference version of everything we did live today. The setup from Session VII lives on [Installing Python](#) and [AI Tools](#).